University of Inland Norway

Faculty of Audiovisual Media and Creative Technologies

Bachelor thesis

Saving the Game: Consistent Offline-Cloud State Synchronization in Survival Video Games

Lagre spillet:
Konsistent synkronisering av lokal- og skybasert tilstand i overlevelsesspill

B2VISIM Bachelor in Game Technology and Simulation

SPIS2900 Bachelor thesis

Hans Alan Whitburn Haugen

May 10, 2026

Abstract

Many survival games store their world entirely on remote servers. This means players cannot progress offline, and if the developer shuts down the online service, the game is lost forever. This thesis considers ways to design a system that supports both offline and online gameplay, and that allows a player to transition from an offline session to an online one without losing progress.

The main point is to log each player action in the form of a command instead of making the changes live on the server immediately. When the player is offline, the commands accumulate in a local queue. As soon as the connection is re-established, these commands are executed on the authoritative server in the same order. The biggest issue is how to reconcile conflicts: for example, if during the period of being offline, the building that a player intended to demolish was destroyed by another player, or the same territory was grabbed by another player, etc. then the system needs to figure out how to handle such situations. This thesis proposes a solution that divides commands into categories: there are commands that can always be executed irrespective of changes, some need a version check, and some still have to be rejected and communicated to the player.

A prototype was built in C++ to validate the design, including a SQLite-backed offline database and a durable command log that survives crashes.

Sammendrag

Mange overlevelsesspill lagrer spillverdenen utelukkende på eksterne servere. Dette gjør det umulig å spille spillet offline, og hvis utvikleren legger ned nettjeneste, så mister spillerne tilgang til spillet for alltid. Denne oppgaven undersøker hvordan et system kan utvikles som gjør at en spiller kan gå mellom en spillverden uten internett og en skybasert spillverden, uten å miste fremgang.

Hovedideen er å loggføre hver spillerhandling i form av en kommando, i stedet for å gjøre endringer direkte på serveren. Når spilleren er offline, samles disse kommandoene i en lokal kø. Så snart forbindelsen er gjenopprettet, spilles de av mot den autoritative serveren i samme rekkefølge. Den største utfordringen er å avgjøre hvordan konflikter skal håndteres: for eksempel, hva skjer når annen spiller har revet ned den samme bygningen eller inntatt det samme territoriet mens en spilte frakoblet? Denne bacherloroppgaven foreslår konflikthåndteringsstrategier som klassifiserer kommandoer etter type: det er kommandoer som alltid utføres uavhengig av hva som har endret seg, andre krever en versjonssjekk, og noen må rett og slett avvises og rapporteres tilbake til spilleren.

En prototype ble bygget i C++ for å validere designet av metoden, inkludert en SQLite-basert offline database og en kommandoliste som overlever at spillet plutselig lukkes.

Acknowledgements

I would like to thank my mentor, Tord Sindre Trøen, for helping me grow as a programmer and for giving me insight into the inner workings of professional game engine development and studio practices. His patience and guidance have strengthened my ability to think critically about code, particularly in the context of code reviews and the broader impact of technical changes. He also introduced me to effective ways of using AI to support code production, while maintaining a critical perspective on the generated results. In addition, he emphasized the importance of seeking help when needed and reaching out to others with prior experience in the systems I work with — advice that has reinforced similar guidance I have received from mentors in the past.

I would also like to thank Maria Esperanza Johnson Ruiz for her continued support throughout the course. I am grateful to Andreas Aasom Brandsnes, who has been an excellent collaborator and a kind friend on past projects. I would further like to thank Håvard Vibeto at the University of Inland Norway, Fredrik Martol at Funcom, and the Funcom Core Engine Team for giving me the opportunity to experience game development within a cutting-edge studio environment.

I would also like to thank my academic advisor at the University of Inland Norway, Ole Einar Flaten, who teaches 3D programming and game engine development—two of my primary areas of interest. His feedback and academic guidance have been invaluable throughout the thesis process, as well as during the internship period from 15 January to 13 May 2026.

Contents

Listings

1 Introduction

This investigation considers a way to develop the persistence system of a modern large-scale survival multiplayer game. The design targets persistent multiplayer worlds shared by thousands of players, and are intended to support the storage of player progression, world state, and faction dynamics. The design is not concerned with ephemeral information such as player position or real-time combat replication; the focus is on long-term persistent data of the kind typically stored in centralized, cloud-hosted SQL databases.

The suggested design is intended to provide a way to support both offline and online functionality. Having the ability to make a game run offline is important, especially on console platforms where access to online multiplayer features typically requires a paid subscription service. A significant portion of console players therefore do not participate in online multiplayer environments (Baker, 2025). Playing with friends should be supported via local cooperative modes, both splitscreen and local listen server setup, while preserving core gameplay mechanics.

This requirement introduces architectural challenges which need to be addressed sensibly. Games on the market which support persistent game state stored exclusively in cloud-based SQL databases assume continuous server connectivity. Supporting offline gameplay necessitates a hybrid architecture in which persistent world state can be stored locally, potentially using embedded database solutions such as SQLite which can later be synchronized with the authoritative cloud-based server infrastructure.

Many gamers prefer to play offline, but might want to connect online as they advance in the persistent world. Collaboration with other players is often necessary in survival games to navigate the harsh environment. Political dynamics and faction-based conflict play a central role in survival games, as groups of players compete for control and influence within the game's persistent world (Céré et al., 2021). Designs that allow users to transition online after initial offline sessions are explored.

The following research statement is investigated:

How can a hybrid offline–online persistence system for survival multiplayer games be designed and evaluated with respect to consistency, conflict resolution, and reconnection performance?

2 Methodology

A design science research approach (Hevner et al., 2004) has been adopted. In Design Science, the main product of the thesis is an artifact; in this project the artifacts are the architectural design of a save system and the actual implemented software prototype of the artifact. Instead of the empirical study, the software prototype is the main end product of the research.

The research is conducted in three phases. The first stage is analysis: it involves analyzing existing, server authoritative persistence systems and identifying the problems associated with playing offline. This analysis is performed in Section 4.

Proposal for second stage design: a hybrid architecture derived from the Command Pattern, with conflict resolution, and showing how the design fits together with online SQL backends and a local SQLite store, is proposed. Design decisions are recorded in Section 5.

The third stage is evaluation, implemented in C++, against two requirements:

1. Correctness, where all parts of the design are covered by automated unit and integration tests, written using the Catch2 framework (Hořeňovský & catchorg contributors, 2023).
2. Performance, where the replay throughput, command log size limits, conflict detection overhead, and the end-to-end cost of a full offline session are measured using a suite of benchmark programs.

Alternative solutions to the research question are briefly discussed in Section 6. The evaluation is described further in Section 7. The results are discussed in Section 8 and future work is touched upon in Section 9. Finally, a conclusion is made in Section 10.

3 Related Work

3.1 Game Industry Approaches

Several commercially released games have addressed aspects of offline-online synchronization, though typically without revealing the underlying mechanisms.

Valheim (Iron Gate Studio, 2021) uses SQLite to store its world state locally on the hosting player's machine. The entire saved game is found in `world.db`. This makes the game fully playable offline and without any cloud dependency, but it provides no mechanism for merging divergent state: if two players host separate copies of the same world while disconnected, there is no supported way to reconcile the differences. This is effectively the snapshot problem described in Section 4 — straightforward for single-player scenarios but unworkable for shared persistent worlds.

Steam Cloud Save (Valve Corporation, 2023) offers file-level synchronization across devices, uploading and downloading save files when a player connects. Conflicts are resolved by taking the most recently modified file, which is a form of last-write-wins and loses any changes made on the other device. This approach is simple and adequate for single-player games, but cannot handle concurrent modifications from multiple players.

World of Warcraft (Blizzard Entertainment, 2004) is one of the earliest and largest-scale examples of a persistent multiplayer world built on a purely server-authoritative model. All world state — character progression, item ownership, guild dynamics — lives exclusively on Blizzard's servers, making offline play neither supported nor meaningful. When Blizzard shut down legacy vanilla servers in 2016, the entire game history of those worlds was lost to players, illustrating the preservation problem discussed in Section 8 with particular clarity.

Blizzard Entertainment's Diablo III (Blizzard Entertainment, 2012) required a persistent connection to the Battle.net infrastructure even for solo gameplay, on the grounds that all character state was stored and validated server-side. At launch in 2012 this meant that server overload made the game completely unplayable despite no multiplayer interaction being involved — an incident widely referred to as "Error 37." This illustrates a concrete failure mode of the purely server-authoritative model: availability of the game depends entirely on availability of the developer's infrastructure, regardless of whether the player is interacting with others.

Many large-scale survival multiplayer games — such as Rust and DayZ — take the same approach and make offline play impossible, requiring a persistent connection to a central server at all times. This sidesteps offline to online persistence synchronization entirely, but at the cost of offline-mode.

The academic distributed systems literature offers more principled solutions, though they come with trade-offs in implementation complexity:

Conflict-free Replicated Data Types (CRDTs) (Shapiro et al., 2011) are mathematical data structures that guarantee eventual consistency without coordination. Any two replicas can be merged in any order and will converge to the same result. CRDTs would eliminate the need for conflict resolution policies entirely, but they require redesigning every game data type to conform to CRDT semantics — a significant architectural investment. They are also better suited to data types like counters and sets than to the complex relational structures (buildings with positions, quest state, faction graphs) that survival games require.

Operational Transformation (OT) (Ellis & Gibbs, 1989), the technique underlying collaborative editing tools such as Google Docs, transforms concurrent operations so that they can be applied in any order while producing a consistent result. The intent-based approach used in this thesis shares similarities with OT: both record operations rather than snapshots. However, OT requires a central server to sequence and transform operations in real time, which is not available during an offline session.

Event sourcing (Fowler, 2005) is a software architecture pattern in which state is derived entirely from an append-only log of events. The command log proposed in this thesis is closely related: both record intent rather than state, and both allow the current state to be reconstructed by replaying the log. The key difference is that event sourcing typically assumes a single authoritative log, whereas this thesis addresses what happens when two divergent logs (client offline, server online) must be merged.

The conflict resolution strategy — particularly entity versioning and optimistic concurrency control — follows the approach described by Bernstein and Goodman (Bernstein & Goodman, 1981), where transactions validate preconditions at commit time rather than acquiring locks upfront. Traditional concurrency control relies on mutexes: a thread or process acquires an exclusive lock before modifying shared state, blocking all other writers until the lock is released. This works well in environments where all participants are continuously connected, but is impractical when a client may be offline for an extended period — holding a lock across a disconnection would stall the entire system. Optimistic concurrency control avoids this by allowing offline progress to proceed freely and deferring conflict detection to the moment of reconnection, making it a natural fit for the hybrid offline–online scenario addressed in this thesis.

The broader challenge of designing software that functions offline and synchronizes when connectivity returns has been studied outside the game domain. Kleppmann et al. (Kleppmann et al., 2019) characterise this as the *local-first* software problem: applications should keep data on the user's own device, remain fully functional without a network connection, and synchronize with collaborators or cloud services when a connection is available. The command log approach proposed in this thesis shares the same underlying goal, applied to the specific constraints of a survival multiplayer game — persistent shared world state, conflict-prone concurrent modifications, and a server that must remain the final authority.

Moll et al. (Moll et al., 2021) propose a quadtree-based protocol for synchronizing game state across server clusters in large-scale multiplayer environments using Named Data Networking. While that work focuses on inter-server synchronization rather than client offline play, it demonstrates that region-based naming and decoupled world representations can reduce the coordination overhead of keeping distributed game state consistent — a concern related to the scalability dimension of the problem addressed here.

4 Typical Challenges in Persistence Systems

Game worlds consist of actors that may own inventory items, buildings, vehicles, navigation markers, abilities, and quest progression. In large-scale online survival games, persistent game state is commonly stored in centralized persistence systems backed by SQL databases hosted on cloud infrastructure.

Such systems typically follow a server-authoritative architecture in which all state-modifying actions are processed through a centralized backend. Player actions are buffered temporarily on the client before being committed to the authoritative server. Multiple server instances may run concurrently, but at any given time only one instance holds authoritative control over a given player or region. Actions are executed as database transactions, providing rollback in the event of failure.

A common design pattern for this kind of system is to apply the Service Locator pattern (Nystrom, 2014b) to abstract the communication layer responsible for generating and dispatching persistence commands. This abstraction is hereafter referred to as the Database Interface (DBI). Rather than issuing raw SQL queries directly from game code, the DBI invokes predefined server-side functions. Each function executes as a transactional unit and can be rolled back if any step fails, so every state-modifying action is encapsulated as an atomic database operation.

A well-designed DBI supports atomicity, consistency, and idempotency. If a network failure occurs, or if the client is uncertain whether a transaction completed, the DBI can safely retry without risking duplicate or inconsistent state changes. An important structural advantage of this abstraction is that the DBI can be swapped out via the Service Locator pattern: an online session routes calls to the remote SQL backend, while an offline session could route them to a local implementation — the implications of which are explored in Section 5.

A SQL-based DBI architecture treats the cloud-hosted SQL database as the single authoritative source of truth for all persistent game state. This provides strong consistency guarantees in fully online environments, but it inherently depends on centralized authority and does not account for state divergence in disconnected scenarios. See the listing below:

Listing 1: DBI interface (abstract base class)

```
class IDatabaseInterface
{
public:
    virtual ~IDatabaseInterface() = default;

    virtual bool CreateBuilding(int32_t playerId, int32_t
        buildingTypeId,
                                float posX, float posY,
                                   float posZ) = 0;
    virtual bool DestroyBuilding(int32_t buildingId) = 0;

    virtual bool TransferItem(int32_t fromPlayerId, int32_t
        toPlayerId,
                              int32_t itemId, int32_t
                                 quantity) = 0;

    virtual bool CompleteQuest(int32_t playerId, int32_t
        questId) = 0;

    virtual bool UpdateFactionRelation(int32_t factionA,
        int32_t factionB,
                                       int32_t relationDelta
                                          ) = 0;
};
```

4.1 Limitations of Server-Authoritative Architectures

A server-authoritative persistence architecture operates under the assumption of continuous connectivity to a centralized cloud infrastructure. While this model ensures strong consistency and transactional integrity in online multiplayer environments, it introduces several limitations when extended to offline or hybrid scenarios.

4.1.1 Dependence on Continuous Connectivity

Because all state-modifying actions are executed as remote transactions, gameplay cannot progress without server access. This dependency prevents offline play entirely and limits deployment flexibility on platforms where online access may be restricted or requires a paid subscription service. In practice, this means that a player who loses their internet connection mid-session — whether due to a router failure, a mobile network dropout, or a server maintenance window — cannot continue playing at all. Any progress made since the last successfully committed transaction is lost, and the client has no mechanism to queue or defer actions for later submission. For players in regions with unreliable infrastructure, or on console platforms where online access requires a paid subscription, the game is effectively inaccessible for extended periods. The hybrid architecture proposed in Section 5 addresses this by decoupling action generation from action transmission.

4.1.2 Absence of Local Persistence Authority

In a purely server-authoritative model, the client does not maintain an authoritative local representation of persistent world state. Although temporary state may exist in memory during a session, durable storage is exclusively managed by the remote infrastructure. This prevents local world progression in disconnected scenarios and complicates support for local cooperative play modes. A concrete consequence is crash sensitivity: if the game process terminates unexpectedly — due to a power failure, an operating system crash, or a forced application quit — any in-memory state accumulated since the last server round-trip is silently lost. The player reconnects to find their progress rolled back, with no indication of how much was undone. A local persistence layer, such as the SQLite-backed DBI described in Section 6.1, would allow progress to survive process termination by writing state to disk before it is confirmed by the server.

4.1.3 Lack of Divergence Handling Mechanisms

A server-authoritative architecture assumes a single linear progression of state changes and does not account for concurrent or divergent modifications occurring in separate environments — for example, an offline client and the live server both modifying the same entity. Consequently, such architectures typically lack:

1. **Version tracking mechanisms** — without a record of which version of an entity a player acted on, it is impossible to determine whether a subsequent action was based on stale information.
2. **Conflict detection strategies** — without version tracking, there is no reliable way to identify when two independent modifications targeted the same entity in incompatible ways.
3. **Deterministic merge policies** — even when a conflict is detected, there must be a defined rule for resolving it; last-write-wins, rejection, or type-specific merging each produce different outcomes for the player.

Without these components, safely synchronizing locally modified state with cloud-based state is not possible. The version-based conflict detection and command classification strategy developed in Section 5.3 directly addresses each of these three gaps.

4.1.4 Scalability vs. Flexibility Trade-off

A centralized server-authoritative design scales well for persistent multiplayer environments but sacrifices flexibility. Extending it toward hybrid offline–online play requires introducing distributed state management — concerns that a purely server-authoritative architecture was not originally designed to address.

The tension is not simply technical: each step toward offline support requires loosening assumptions that the rest of the system depends on. Allowing local writes means accepting that two versions of the same entity can exist simultaneously. Accepting divergent state means requiring conflict resolution. Requiring conflict resolution means classifying every operation by its merge semantics. These are compounding concerns, and retrofitting them onto an architecture built around a single authoritative source of truth is substantially more difficult than designing for them from the outset.

5 Proposed Hybrid Persistence Architecture

5.1 Command-Based State Replication

To enable offline progression while preserving compatibility with a server-authoritative model, this thesis proposes a command-based persistence mechanism inspired by the Command Pattern as described in Design Patterns: Elements of Reusable Object-Oriented Software (Gamma et al., 1994) and further discussed in Game Programming Patterns (Nystrom, 2014a).

Rather than directly modifying persistent state in the cloud, player actions are encapsulated as serializable command objects. Each command represents an intention to modify game state (e.g., create building, transfer item, complete quest) along with the necessary parameters to execute the modification.

During offline gameplay, commands are appended to a durable local command log instead of being transmitted immediately to the server. When connectivity is restored, the queued commands are transmitted sequentially to the authoritative backend infrastructure and executed as transactional operations.

This approach introduces several advantages:

1. Commands are naturally serializable and can be stored locally.
2. Replay of commands enables deterministic reconstruction of state.
3. The command log provides a history of offline actions.
4. Existing server-side transactional guarantees can remain unchanged.

By storing intent rather than derived state, the system avoids directly merging divergent world snapshots and instead synchronizes through ordered execution of state-modifying operations. See the listing below:

Listing 2: Command structure and local command log

```
struct Command
{
    std::string type;            // e.g. "CreateBuilding", "
        ResourceDelta"
    int64_t     timestamp;       // UTC milliseconds at time
         of action
    std::string idempotencyKey;  // UUID to detect duplicate
         replays
    std::string entityId;        // ID of the affected
        entity
    int32_t     expectedVersion; // entity version at
        command creation
    int32_t     delta;           // signed delta for
        commutative operations
};

class CommandLog
{
public:
    void                 Append(const Command& cmd);
    std::vector<Command> GetPending() const;
    void                 MarkFlushed(const std::string&
        idempotencyKey);
    void                 MarkFlushedBatch(
                             const std::vector<std::string>&
                                 keys);
    std::size_t          PendingCount() const;

private:
    std::vector<Command> m_pending;
};
```

5.2 Implementation considerations

This system must be designed with integrity guarantees in mind. Since commands are generated client-side and stored locally, a malicious client could potentially forge or alter commands before transmission. Every action must therefore be validated on the server before it is committed, and any command that produces an invalid or impossible state transition must be rejected. Conflict detection, entity versioning, and per-type resolution policies are addressed in the following subsection.

Beyond conflict handling, two additional concerns must be managed. First, ordering guarantees: the command log must preserve the sequence in which actions were taken offline, and the server must replay them in that order to ensure deterministic results. Second, idempotency and duplicate detection: if a replay attempt fails partway through or a command is transmitted more than once, the server must be able to identify and skip already-applied commands. The idempotency key stored on each command serves this purpose.

A remaining open concern is command dependency on server-side state that changed during the offline session. If an offline command assumed the existence of an entity that was deleted by another player while the client was disconnected, the command cannot be applied even if its version check would otherwise pass. Handling such cases requires pre-condition checks on the server that go beyond version comparison, and the rejection manifest provides the mechanism for communicating these failures back to the client.

5.3 Conflict Resolution Strategy

When offline commands are replayed against the authoritative server, conflicts arise when the server state has diverged from the state against which those commands were originally generated. A command that was valid at the time of creation may produce an invalid or inconsistent state transition when applied to an evolved world.

5.3.1 Conflict Detection via Entity Versioning

Each persistent entity (e.g. a building, an inventory slot, or a faction) is assigned a monotonically increasing version number that is incremented with every committed state change. When a command is generated offline, the version of each affected entity is recorded alongside the command parameters. Upon replay, the server compares the recorded version against the current entity version. A mismatch indicates that the entity was modified after the command was issued, signalling a potential conflict.

This mechanism mirrors optimistic concurrency control (OCC) in traditional database systems, where transactions proceed without locking but are validated at commit time (Bernstein & Goodman, 1981).

5.3.2 Command Classification

Not all version mismatches require rejection. Commands can be classified by their conflict behaviour:

1. **Idempotent commands** — Quest completion, achievement unlocks, and similar one-time flags. These produce the same result regardless of how many times they are applied and can be safely replayed unconditionally.
2. **Commutative commands** — Additive operations such as resource collection or faction reputation deltas. Because the order of application does not affect the final value, these can be merged without conflict by summing the deltas.
3. **Non-commutative commands** — Destructive or positional operations such as demolishing a building or placing a structure on a specific grid

cell. These depend on preconditions that may have been invalidated by independent server-side changes.

5.3.3 Resolution Policies

For each conflict class, a deterministic resolution policy is applied server-side before any command is committed. See listing below:

Listing 3: Server-side command validation and conflict resolution

```
ReplayResult ReplayCommand(const Command& cmd, Database& db,
                           RejectionManifest& manifest)
{
    // Idempotent: already applied, skip silently
    if (db.IsAlreadyApplied(cmd.idempotencyKey))
        return ReplayResult::Duplicate;

    // Commutative: apply delta regardless of entity version
    if (cmd.type == "ResourceDelta" || cmd.type == "
    ReputationDelta")
        return db.ApplyDelta(cmd);

    // Non-commutative: validate entity state before
    committing
    Entity entity = db.GetEntity(cmd.entityId);

    if (!entity.exists)
    {
        manifest.push_back({cmd, ReplayResult::Rejected});
        return ReplayResult::Rejected;
    }

    if (entity.version != cmd.expectedVersion)
    {
        manifest.push_back({cmd, ReplayResult::Conflict});
        return ReplayResult::Conflict;
    }

    return db.Commit(cmd);
}
```

5.3.4 Rejection Manifest

Commands that fail validation are not silently discarded. Instead, they are collected into a rejection manifest that is returned to the client after the replay pass completes. The manifest records the command type, the reason for rejection, and the current server-side state of the affected entity. This allows the game to inform the player of which offline actions could not be applied and why, enabling the client to present meaningful feedback rather than silently losing progress.

5.4 Command Dispatch and SQL Execution

Once `ReplayCommand` determines that a command passes validation, it calls `db.Commit(cmd)`. The commit step dispatches on the command's `type` field and invokes the corresponding method on the DBI, which translates the operation into one or more SQL statements. The mapping is straightforward: each command type corresponds directly to a single DBI method, and each DBI method is responsible for the SQL that implements the game-domain operation it represents.

Listing 4: Server-side command dispatch to DBI methods

```
ReplayResult Database::Commit(const Command& cmd)
{
    if (cmd.type == "CreateBuilding")
        dbi.CreateBuilding(cmd.entityId, cmd.delta, 0, 0, 0)
            ;
    else if (cmd.type == "DestroyBuilding")
        dbi.DestroyBuilding(cmd.entityId);
    else if (cmd.type == "TransferItem")
        dbi.TransferItem(cmd.entityId, cmd.delta);
    else if (cmd.type == "CompleteQuest")
        dbi.CompleteQuest(cmd.entityId, cmd.delta);
    else if (cmd.type == "ResourceDelta" ||
             cmd.type == "ReputationDelta")
        dbi.UpdateFactionRelation(cmd.entityId, cmd.delta);

    RecordApplied(cmd.idempotencyKey);
    IncrementVersion(cmd.entityId);
    return ReplayResult::Committed;
}
```

Each DBI method executes a specific SQL pattern chosen to preserve the correctness properties of that command class:

Command type	DBI method	SQL pattern
`CreateBuilding`	`CreateBuilding`	`INSERT INTO buildings`
`DestroyBuilding`	`DestroyBuilding`	`DELETE FROM buildings WHERE id = ?`
`TransferItem`	`TransferItem`	`BEGIN; SELECT / UPDATE / UPSERT; COMMIT`
`CompleteQuest`	`CompleteQuest`	`INSERT OR IGNORE INTO completed_quests`
`ResourceDelta`	`UpdateFactionRelation`	`INSERT ... ON CONFLICT DO UPDATE SET`
`ReputationDelta`	`UpdateFactionRelation`	`relation = relation + excluded.relation`

The SQL patterns are not arbitrary. `INSERT OR IGNORE` for quest completion makes the operation inherently idempotent at the database level — a duplicate replay of the same quest completion is silently discarded even if the idempotency check at the application level somehow fails. The UPSERT pattern for resource and reputation deltas (`ON CONFLICT DO UPDATE SET value =`

`value + excluded.value`) accumulates the delta additively, which is the correct merge behaviour for commutative operations regardless of how many times the command is replayed. The explicit `BEGIN/COMMIT` transaction wrapping `TransferItem` ensures that the debit from the sender and the credit to the recipient are atomic — either both succeed or neither does, preventing inventory from being created or destroyed by a partial failure.

5.5 Integration Possibilities

In a server-authoritative game, every action that should persist goes through the DBI. By extending the DBI into a factory that produces command objects, the system can route actions in two ways depending on connectivity state: online commands are transmitted to the server immediately, while offline commands are serialized and stored on disk, deferred until the player reconnects.

The routing decision is straightforward to implement. A factory function checks connectivity and returns the appropriate DBI implementation:

Listing 5: Connectivity-aware DBI factory

```
std::unique_ptr<IDatabaseInterface> CreateDBI(bool isOnline)
{
    if (isOnline)
        return std::make_unique<RemoteSQLDBI>(serverConfig);
    else
        return std::make_unique<SQLiteDBI>("offline.db");
}
```

Game code calls DBI methods without knowing which backend is active. When connectivity is restored, the engine drains the durable command log by calling `ReplayAll`, which submits each queued command to the server in order and returns a `RejectionManifest` listing any commands that could not be applied.

Figure 1 shows the overall data flow.

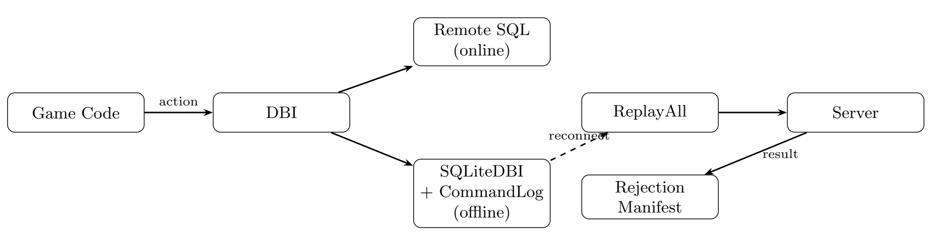

Figure 1: Hybrid persistence architecture: game code interacts only with the DBI; the active implementation is swapped at runtime depending on connectivity.

5.6 Prototype Implementation

A self-contained C++ prototype of the proposed architecture was developed alongside this thesis and is available at:

https://github.com/alanhaugen/SPIS2900-Bachelor

The prototype is located in the `app/` directory and is built with CMake. It depends only on Catch2, which is fetched automatically at configure time, and a system SQLite3 library. The implementation is organized as two libraries:

`offline_sync_lib` contains the core persistence abstractions:

- `IDatabaseInterface` — the abstract DBI used by game code, with virtual methods for all persistent game actions.
- `Command` and `CommandLog` — the in-memory command queue described in Section 5.1.
- `Database` — a lightweight in-memory entity store used for testing, with version tracking and idempotency key recording.
- `ReplayCommand` / `ReplayAll` — the conflict-resolution algorithm from Section 5.3, including the `RejectionManifest` type.

`offline_sync_sqlite_lib` contains the SQLite-backed implementations:

- `SQLiteDBI` — a concrete implementation of `IDatabaseInterface` that stores game state in a local SQLite file. It maintains game-domain tables for buildings, item inventories, completed quests, and faction relations. Item transfers execute inside an explicit SQLite transaction and are rejected atomically if the sender has insufficient inventory. Quest completion uses `INSERT OR IGNORE` to preserve idempotency. Faction relation updates use an UPSERT to accumulate signed deltas.
- `SQLiteCommandLog` — a durable replacement for the in-memory `CommandLog` that writes each command to a SQLite file before returning. The `idempotency_key` column carries a `UNIQUE` constraint so that a double-append is silently discarded at the database level. Commands survive process restarts and can be recovered on next open.

The prototype includes 31 Catch2 unit tests covering the command log, idempotency, commutative merging, non-commutative conflict detection, SQLite persistence, and a full offline-session integration scenario.

6 Alternative State Replication Persistence Architectures

Other simpler alternatives to the Command-Based State Replication have been considered and will be described and discussed below. Some of the problems with the alternative architectures revolve around a lack of functionality for solving divergent state (see section 6.1) and implementation complexity (see section 6.2).

6.1 Local Persistence via SQLite

A direct approach to enabling offline functionality is to extend the Database Interface (DBI) abstraction with a complete SQLite-backed implementation. Rather than communicating with remote SQL servers, the DBI would execute equivalent operations against a locally embedded SQLite database during offline sessions.

SQLite (SQLite Consortium, 2024) is an embedded, serverless, ACID-compliant relational database engine that operates as a lightweight C/C++ library. Unlike other SQL implementations, it does not require a separate server process; instead, it runs within the host application and stores data in a single file on disk. This makes it well-suited for offline persistence in desktop and console environments.

By preserving the DBI abstraction layer, the system can maintain a consistent programming interface for persistence operations regardless of connectivity state. When offline, DBI calls would be routed to the SQLite backend. When online, calls would be directed to the existing SQL infrastructure.

This design minimizes disruption to the gameplay code, as the persistence boundary remains unchanged. However, it introduces challenges related to schema compatibility, transaction semantics, and synchronization between the local SQLite instance and the authoritative cloud database.

A deeper limitation is that a SQLite-backed DBI alone is not sufficient for synchronization. The SQLite DBI is a *readable state store* — it answers the question "what does the world look like right now?" and allows the game to display inventory, buildings, and quest progress during an offline session. What it does not provide is a record of what *changed* during that session. On reconnection, the server has no way to know which operations the player performed; it can only observe that local and remote state have diverged. Merging two divergent snapshots requires structural diffing of every entity, which becomes increasingly error-prone as the world model grows more complex.

This is why the command log is a necessary complement rather than an optional addition. The command log is a *pending change queue* — it records each action as it is taken so that the exact sequence of player intent can be replayed on the server. After a successful replay the log is drained; the SQLite state store is not. The two components answer different questions and cannot substitute for each other: removing the command log leaves no replay record, while removing the SQLite DBI leaves the game unable to read any state while offline.

6.2 ORM (Object-Relational Mapping)

An alternative architectural approach considered was the introduction of an Object–Relational Mapping (ORM) system. An ORM provides a layer that maps in-memory C++ objects directly to relational database structures, allowing persistent entities to be stored and retrieved without writing explicit SQL logic. In such a system, game objects (e.g., inventory items, buildings, or quests) would be automatically translated into database rows and reconstructed from relational data.

ORM frameworks are common in higher-level languages such as Python and Java, where reflection and dynamic typing simplify runtime type inspection and schema generation. In contrast, implementing a fully featured ORM in C++ presents additional complexity due to the lack of built-in reflection and the need for explicit type registration and serialization logic.

The deeper problem, however, is a fundamental mismatch between the ORM model and the command-based architecture proposed in this thesis. An ORM operates on *state*: it tracks which object fields have changed and generates SQL to persist the new snapshot. The command log operates on *operations*: it records what the player did, not what the resulting world state looks like. These two models are philosophically opposed. If an ORM were used as the persistence layer, it would continuously collapse player actions into state diffs, discarding precisely the operation-level intent that the command log needs to preserve for server-side replay. The server would receive a snapshot of what changed, not a replayable record of how it changed — making conflict classification by command type impossible.

By contrast, the DBI abstraction already provides the backend-agnosticism that an ORM is typically chosen for: game code interacts with a stable interface and is unaware of whether calls are routed to a remote SQL server or a local SQLite file. The difference is that the DBI exposes game-domain operations (`TransferItem`, `CompleteQuest`) rather than generic object persistence, which is exactly the granularity needed to classify commands for conflict resolution. The DBI is, in effect, a domain-specific alternative to a generic ORM — and for this use case, the domain-specific design is the more appropriate choice.

7 Evaluation

The prototype was evaluated along two dimensions: correctness, verified through automated testing, and performance, measured through a benchmark suite run on the same hardware used for development (Apple M-series, macOS, Release build with compiler optimisations enabled).

7.1 Correctness

The implementation is covered by 31 Catch2 unit and integration tests organised into five categories: command log behaviour (append, ordering, flush), idempotency (duplicate commands produce no side effects), commutative merging (resource and reputation deltas apply regardless of version mismatch), non-commutative conflict detection (version mismatch and missing entities are caught and reported), and a full offline session integration test that exercises all result types in a single replay pass. All 31 tests pass.

7.2 Replay Throughput

The first benchmark measures how quickly the in-memory replay engine can process a command log of increasing size, using one unique entity per command to isolate the cost of the replay loop itself from conflict resolution overhead.

Commands	Time (ms)	Throughput (cmd/s)	Applied
100	0.02	4,907,975	72
500	0.09	5,780,347	368
1,000	0.16	6,448,160	734
5,000	0.81	6,196,747	3,752
10,000	1.44	6,957,533	7,418
50,000	9.36	5,340,668	37,018

Throughput is consistent at approximately 5–7 million commands per second across all log sizes, confirming that replay scales linearly with the number of commands (Figure 2). A log of 10,000 commands — large by the standards of a single offline session — replays in under 2 milliseconds, well below any perceptible threshold. The applied count is lower than the total because 30% of commands are commutative and increment the entity version, causing subsequent non-commutative commands targeting the same entity to detect a version mismatch. This reflects realistic intra-session behaviour when a player modifies the same entity more than once.

During development, this benchmark identified a quadratic performance issue in the initial implementation. The original `ReplayAll` called `MarkFlushed` once per resolved command, each of which performed a linear scan of the pending vector. This produced $O(n^2)$ behaviour: 50,000 commands took 2,939 ms before the fix. The fix — collecting all resolved keys and removing them in a single pass — reduced that to 9 ms, a 313-fold improvement.

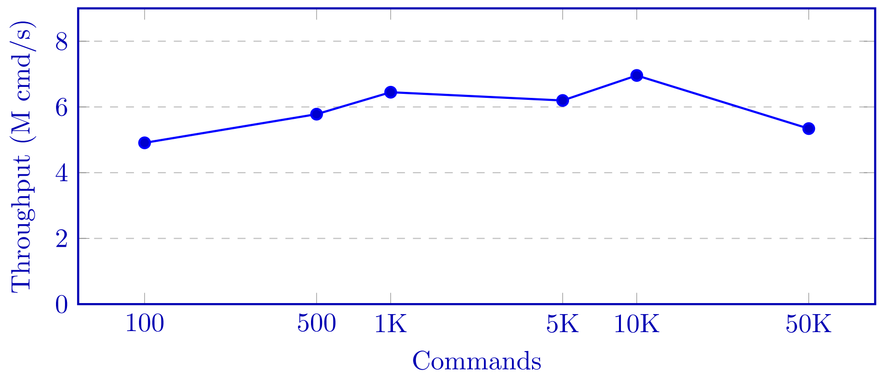

Figure 2: Replay throughput remains stable at 5–7 M cmd/s regardless of log size.

7.3 Command Log Backend Comparison

The second benchmark compares the in-memory `CommandLog` against the SQLite-backed `SQLiteCommandLog` for append and retrieval operations.

Commands	Mem. Append (ms)	SQL Append (ms)	Mem. Get (ms)	SQL Get (ms)
100	0.00	0.33	0.00	0.03
1,000	0.01	3.35	0.00	0.20
5,000	0.06	19.96	0.02	1.06
10,000	0.13	33.57	0.03	1.82

SQLite append is approximately 250 times slower than in-memory (33 ms vs 0.13 ms for 10,000 commands), as shown in Figure 3; this is expected given that SQLite writes to disk with ACID guarantees. In absolute terms the overhead is still acceptable for an offline session: a player generating 10,000 actions — far more than a typical session — would incur roughly 33 ms of storage overhead. Retrieval via `GetPending` is fast in both backends; at 10,000 commands SQLite takes under 2 ms.

7.4 Conflict Detection Overhead

The third benchmark holds the command count fixed at 10,000 and varies the proportion of commands that are guaranteed to produce a version conflict, from 0% to 100%.

Conflict rate	Time (ms)	Throughput (cmd/s)	Applied
0%	1.10	9,130,686	3,246
25%	0.92	10,898,185	3,213
50%	0.93	10,696,617	3,175
75%	0.98	10,163,025	3,085
100%	0.89	11,230,176	2,912

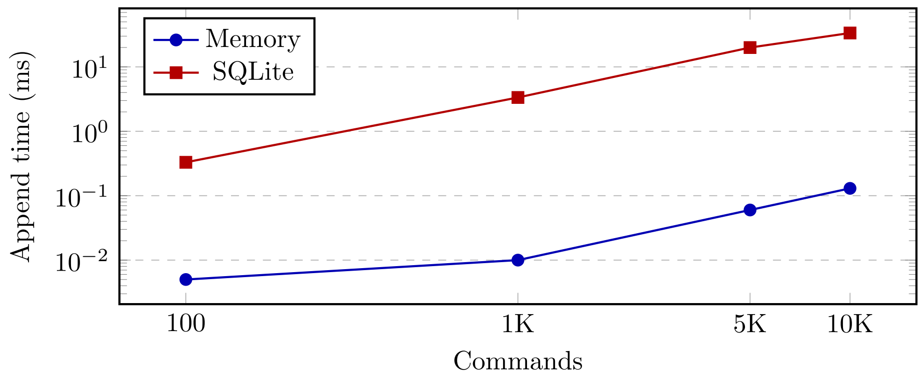

Figure 3: SQLite append is ∼250× slower than in-memory due to ACID disk writes; both scale linearly.

Throughput varies by less than 20% across the full range of conflict rates (Figure 4). Higher conflict rates are marginally faster because rejected commands skip the database write step. This confirms that the version check and classification logic add negligible overhead and that the system degrades gracefully under adversarial conflict conditions.

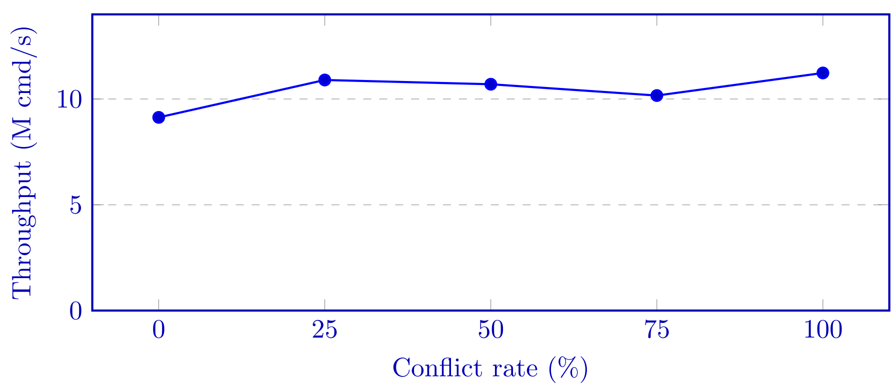

Figure 4: Throughput remains within 20% across all conflict rates, confirming conflict detection adds negligible overhead.

7.5 Full Offline Session Round-Trip

The fourth benchmark measures the complete cost of an offline session using the SQLite command log: appending commands to the durable log, retrieving them, and replaying against the in-memory database.

Commands	Append (ms)	GetPending (ms)	ReplayAll (ms)	Total (ms)
100	0.30	0.02	0.15	0.47
1,000	2.86	0.17	0.81	3.84
5,000	15.26	0.84	3.09	19.19
10,000	30.42	1.70	6.29	38.40

A full session of 10,000 commands completes in approximately 38 ms end-to-end (Figure 5). SQLite append accounts for roughly 80% of that time; actual replay takes only 6 ms. All three phases scale linearly with command count. The results indicate that reconnection latency attributable to synchronization is unlikely to be noticeable in practice for sessions of realistic size.

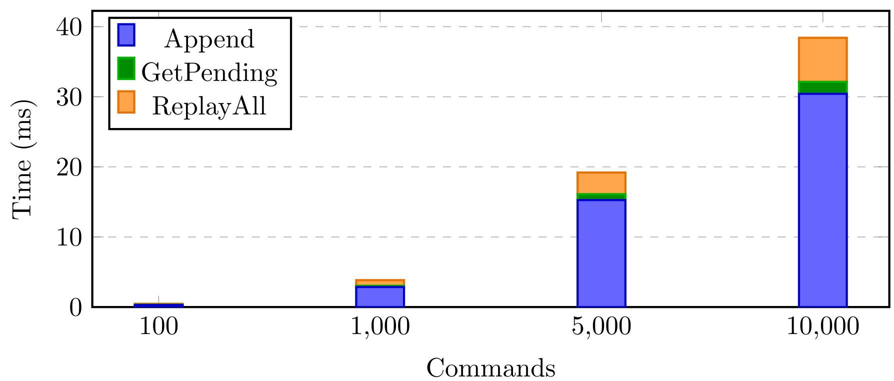

Figure 5: Full offline session cost broken down by phase; SQLite append dominates at ∼80% of total time.

7.6 Command Log Disk Footprint

The fifth benchmark measures how much disk space the SQLite-backed command log consumes as a function of log size. Commands are generated with UUID-length idempotency keys (36 characters) to reflect realistic production identifiers. File size is measured after the database connection is closed, including any WAL file that was not checkpointed.

Commands	File size (KB)	Bytes per command
100	32.0	327.7
500	80.0	163.8
1,000	140.0	143.4
5,000	628.0	128.6
10,000	1,248.0	127.8

The per-command cost stabilizes at approximately 128 bytes for logs of 1,000 commands or more. This overhead has two components: the raw data content of each row (idempotency key at 36 bytes, type string at approximately 13 bytes, entity ID, two integers, and timestamp — roughly 75 bytes in total) and SQLite's structural overhead from the B-tree page layout and the secondary index on the `idempotency_key` column, which adds approximately 50 bytes per row.

The high per-command cost at small log sizes (327 bytes at 100 commands) is a consequence of SQLite's fixed page size of 4 KB: even a single-row database occupies at least two pages — one for the data table and one for the unique index. Once the log grows large enough to fill pages efficiently the overhead drops to the steady-state figure.

In practical terms, a 1,000-command offline session — already a large session by most standards — occupies 140 KB. A session of 10,000 commands occupies approximately 1.2 MB (Figure 6). These figures are negligible on any modern storage medium and confirm that disk space is not a limiting factor for the command log approach.

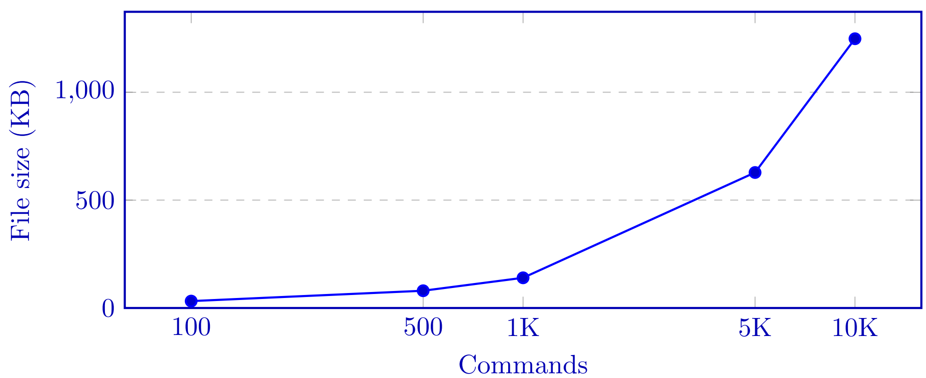

Figure 6: Command log file size grows linearly with command count; 10,000 commands occupy ∼1.2 MB.

8 Discussion

8.1 Architectural Trade-offs

Persistent state management in online multiplayer games can be implemented using a wide range of architectural strategies, each with different trade-offs between consistency, flexibility, and implementation complexity.

A purely server-authoritative architecture provides strong consistency by maintaining a single source of truth, but it cannot support offline play and becomes unavailable the moment a player loses connectivity. This reflects the CAP theorem (Gilbert & Lynch, 2002), which states that a distributed system cannot simultaneously guarantee consistency, availability, and partition tolerance. A server-authoritative model prioritises consistency over availability: when the network is partitioned (the client is offline), the system refuses to accept writes rather than risk divergence. The hybrid design in this thesis relaxes that choice — writes are accepted locally during a partition and validated at reconnection — which means temporarily trading strict consistency for availability, with conflict resolution as the mechanism for restoring consistency afterwards.

The command-based approach proposed here sits between these two extremes. By recording player intent as serialized commands rather than storing divergent state snapshots, synchronization occurs through ordered replay rather than structural merging. This preserves the existing transactional guarantees of the backend while keeping the local state representation minimal. Compared to adopting CRDTs or operational transformation, it also requires substantially less redesign of the existing persistence layer, at the cost of requiring careful conflict classification and server-side validation.

8.2 Evaluation Findings

The benchmark results support the architectural claims in several concrete ways.

The replay throughput experiments show that processing a command log of 10,000 entries takes under 2 ms in the in-memory case and under 7 ms including SQLite retrieval. These figures suggest that reconnection latency attributable to the synchronization mechanism itself is unlikely to be perceptible in practice. The data transmitted on reconnection scales with the number of actions taken offline rather than with the size of the world, which is a meaningful advantage over snapshot-based approaches in large persistent worlds.

The conflict rate experiment provides a particularly useful result: throughput varies by less than 20% across the full range of conflict rates, from 0% to 100%. This means the system does not degrade meaningfully under adversarial conditions, and conflict detection can be considered a negligible overhead rather than a potential bottleneck.

The benchmarking process also revealed a design flaw in the initial implementation. The original `ReplayAll` called `MarkFlushed` individually for each resolved command, each of which performed a linear scan of the pending vector. This produced $O(n^2)$ behaviour that was not apparent from code review alone: 50,000 commands took nearly 3 seconds before the fix and 9 ms after. This illustrates the value of performance testing as part of the design process, not just as a post-hoc validation step.

8.3 Scalability Considerations

The benchmarks characterise the performance of a single-client replay on one machine, which is a necessary but not sufficient basis for scalability claims about a large-scale multiplayer game. Several factors outside the scope of the prototype would affect performance in a production environment.

First, network latency is not modelled. Transmitting a command log over the network and waiting for server acknowledgements would dominate the reconnection time for large logs, dwarfing the sub-millisecond replay costs measured here. Second, the server-side validation logic — checking entity versions and preconditions — was evaluated against an in-memory store. Against a real SQL database under concurrent load from many reconnecting players, validation throughput would depend heavily on database indexing, connection pooling, and transaction isolation levels. Third, the benchmarks use a synthetic workload with uniformly distributed entity access. Real player behaviour may be bursty and spatially clustered, which could affect conflict rates and cache behaviour in ways not captured by the synthetic experiments.

These limitations do not undermine the core design, but they do mean that the benchmark results should be treated as indicative of single-client algorithmic performance rather than a prediction of production capacity.

8.4 Limitations and Threats to Validity

The conflict resolution strategy handles the three command classes identified in this thesis — idempotent, commutative, and non-commutative — but does not cover all edge cases that a production system would encounter. The most significant gap is commands that depend on entities which were deleted server-side during the offline session: these are caught by the existence check in `ReplayCommand`, but the player receives only a rejection notification with no automatic recovery path. A production system would need policies for how to surface and resolve these failures in a way that makes sense within the game's design.

The evaluation workload was generated synthetically using a uniform random distribution over entities and command types. It is not known whether this distribution is representative of real player behaviour, which likely varies by game genre, session length, and player experience level. Collecting telemetry from real offline sessions would allow future work to calibrate the benchmark parameters and test the system under more realistic conditions.

Entity versioning relies on an implicit assumption that the server is the sole authority over version increments and that the counter is strictly monotonic. Under these conditions accidental version alignment is not possible: because versions only increase, any server-side modification to an entity will produce a version strictly higher than what the client recorded offline, and the mismatch will be detected correctly. However, this guarantee depends entirely on correct server-side implementation — a bug that modifies an entity without incrementing its version would silently pass the version check, allowing a potentially conflicting command to be committed. An alternative that does not carry this assumption is to store a cryptographic hash of the entity state alongside the version number. Because a hash reflects the actual content of the entity rather than a counter, it detects state changes even if the version increment is accidentally omitted. The trade-off is cost: hashing the full entity state on every command validation is more expensive than an integer comparison, and under reconnection load from many concurrent clients the overhead compounds. For a trusted server-authoritative environment the integer counter is sufficient; the hash approach is better suited to settings where the integrity of the version counter itself cannot be fully guaranteed.

8.5 Game Preservation

Beyond the technical design, offline functionality has an important consequence for the long-term availability of games as cultural artefacts. Games that depend on continuous server connectivity cease to function when the developer shuts down the online service, whether due to bankruptcy, acquisition, or commercial decision. Building in offline capability means that the game remains playable independently of the publisher's infrastructure, which is directly relevant to current policy debates around game preservation (Ondruška et al., 2026).

8.6 Financial Considerations

Offline capability also has practical consequences for both players and developers that extend beyond technical design.

From the player perspective, accessing online multiplayer on console platforms typically requires a paid subscription — PlayStation Plus, Xbox Game Pass Core, or Nintendo Switch Online. An offline-capable design removes this financial barrier (Baker, 2025), and broadens the reachable audience without requiring any change to the online multiplayer experience for players who do subscribe.

From the developer perspective, running always-on server infrastructure for a large-scale multiplayer game is expensive. Servers must remain provisioned and operational at all times, regardless of how many players are actually connected, to guarantee immediate availability when players log in. A hybrid architecture reduces this pressure: players who are offline generate no server load, and infrastructure can be scaled closer to active concurrent usage rather than peak theoretical demand.

9 Future Work

Several directions for future investigation emerge from the limitations identified in this thesis.

Telemetry-calibrated benchmarks. The evaluation used synthetically generated command logs with uniform random distributions. Collecting real telemetry from offline gameplay sessions would allow the benchmark parameters — entity access distributions, session lengths, conflict rates — to be calibrated against actual player behaviour. This would give a more reliable picture of expected performance and surface corner cases that synthetic workloads may miss.

Production server-side scaling. The server-side replay was evaluated against an in-memory entity store. A production deployment would run validation against a live SQL database under concurrent load from many clients reconnecting simultaneously. Future work should measure how the validation pipeline scales with database contention, test appropriate transaction isolation levels, and explore batching strategies for the server-side replay endpoint.

Network latency modelling. Replay throughput was measured locally, but in practice the command log is transmitted over a network before server-side validation begins. A more complete evaluation would model round-trip latency and measure how command log size affects reconnection time across different network conditions.

Recovery paths for rejected commands. The current design returns a rejection manifest to the client but does not specify what the game should do with it. A production system would need player-facing flows for communicating which offline actions were declined and offering resolution options — for example, refunding consumed resources or offering equivalent substitute items. Designing these recovery policies in a way that is both technically correct and satisfying to players is a non-trivial game-design problem.

Command dependency graphs. Some commands may depend on earlier commands in the same offline log — for example, building a structure and then upgrading it. If the first command is rejected, the second is also invalid regardless of its version check. Modelling explicit command dependencies would allow the replay engine to reject dependent chains early and produce more informative rejection manifests.

10 Conclusion

This thesis investigated the following research question: How can a hybrid offline–online persistence system for survival multiplayer games be designed and evaluated with respect to consistency, conflict resolution, and reconnection performance?

The starting point was a server-authoritative persistence architecture in which all state changes are executed as transactional SQL operations against a centralized cloud backend. While robust in fully online environments, this model assumes continuous connectivity and does not support offline progression or hybrid play modes. Enabling offline functionality therefore requires architectural changes that preserve data integrity while allowing temporary divergence of state.

Two alternative approaches were evaluated before arriving at the proposed design. Direct local database mirroring through SQLite provides a familiar persistence interface during offline sessions but does not itself address how divergent state is reconciled upon reconnection. An object–relational mapping layer was also considered but rejected due to a fundamental mismatch: an ORM tracks object state rather than operations, which is incompatible with the command-based replication model.

The proposed solution is a command-based replication model. Rather than synchronizing complete world snapshots, the system records player intent as serializable commands that can be replayed deterministically against the authoritative server. This approach addresses the three dimensions of the research question as follows.

Consistency is maintained by treating the cloud server as the single authoritative source of truth. Offline commands are not committed until they have been validated and replayed server-side. The transactional guarantees of the SQL backend are preserved throughout.

Conflict resolution is handled through a combination of entity versioning, command classification, and per-type resolution policies. Idempotent commands are replayed unconditionally; commutative operations such as resource deltas are merged by summation; and non-commutative operations are validated against current server state before being accepted or rejected. Commands that cannot be applied are returned to the client in a rejection manifest rather than being silently discarded.

Reconnection performance was evaluated through a suite of benchmarks on a single development machine. A log of 10,000 commands replays in under 2 ms in memory and under 40 ms end-to-end including SQLite persistence. Conflict detection adds less than 20% overhead regardless of conflict rate, and the command log file remains under 1.2 MB at 10,000 commands. These results indicate that the synchronization mechanism itself is unlikely to be a perceptible bottleneck during reconnection for sessions of realistic size.

The approach introduces its own requirements: commands must be deterministic and replay-safe, ordering must be preserved across the command log, and server-side validation logic must cover all command types to prevent exploitation. These constraints are manageable within a server-authoritative architecture but would require careful engineering in a production system.

Overall, the results suggest that a command-based synchronization model provides a practical and extensible foundation for hybrid offline–online persistence in survival video games, and that consistent synchronization can be achieved without fundamentally redesigning a server-authoritative framework.

Glossary

ACID Atomicity, Consistency, Isolation, Durability — the four properties that guarantee database transactions are processed reliably even in the event of errors, power failures, or concurrent access.

Atomicity Atomicity is a fundamental database principle ensuring transactions are treated as single, indivisible "all-or-nothing" units.

B-tree A self-balancing tree data structure that keeps data sorted and allows searches, insertions, and deletions in logarithmic time; the primary index structure used internally by SQLite.

Battle.net Blizzard Entertainment's online gaming platform and authentication infrastructure, required for connectivity in all modern Blizzard titles.

Catch2 A header-based C++ unit testing framework that provides test case macros, assertions, and test discovery without requiring a separate test runner.

CMake A cross-platform build system generator that produces native build files (Makefiles, Xcode projects, Visual Studio solutions) from a platform-independent description.

command log A durable, ordered sequence of command objects representing player actions recorded during an offline session; the primary artifact used to synchronize client state with the authoritative server on reconnection.

Command Pattern A software design pattern in which each action or request is encapsulated as a self-contained object, enabling serialization, queuing, logging, and replay of operations.

commutative operation An operation whose result is independent of the order in which it is applied relative to other operations; in this thesis, additive operations such as resource and reputation deltas that can be merged by summation.

CRDT Conflict-free Replicated Data Type — a mathematical data structure designed so that any two replicas can be merged in any order and will converge to the same result, guaranteeing eventual consistency without coordination.

DBI Database Interface — the abstract layer between game code and the underlying persistence backend; all state-modifying game actions are routed through the DBI, which can be swapped at runtime to target different backends.

Design Science Research A research methodology in which the primary output is an artifact — such as an architectural design or a working software prototype — rather than an empirical study; introduced by Hevner et al. (2004).

entity versioning A mechanism that assigns each persistent entity a monotonically increasing version number, incremented on every committed state change; used to detect whether an entity was modified between the time a command was generated and the time it is replayed.

event sourcing A software architecture pattern in which all changes to application state are stored as an append-only sequence of events, allowing the current state to be reconstructed by replaying the event log from the beginning.

idempotency The property of an operation whereby applying it multiple times produces the same result as applying it once; essential for safe retry logic and duplicate detection in distributed systems.

idempotency key A unique identifier (typically a UUID) attached to each command; the server uses it to detect and silently skip commands that have already been applied, enabling safe re-transmission after network failures.

listen server A listen server is a multiplayer game architecture where one player's computer acts as both the server and a client simultaneously. It allows for easy, cost-effective, local hosting without dedicated hardware, but the host's machine requires strong performance to handle both roles, and the session ends if the host leaves..

Monotonically Monotonically describes a process, function, or sequence that changes in only one direction—consistently increasing or consistently decreasing without reversing direction.

Mutex lock In computer science, a lock or mutex (from mutual exclusion) is a synchronization primitive that prevents state from being modified or accessed by multiple threads of execution at once.

non-commutative operation An operation whose result depends on the order in which it is applied; in this thesis, destructive or positional operations such as demolishing a building or placing a structure that must be validated against current server state before being accepted.

OCC Optimistic Concurrency Control — a concurrency strategy in which transactions proceed without acquiring locks, but validate that their preconditions still hold at commit time; conflicts cause the transaction to be rejected rather than blocked.

Operational Transformation A technique for reconciling concurrent edits by transforming operations so that they can be applied in any order while producing a consistent result; the mechanism underlying collaborative editing tools such as Google Docs.

ORM Object-Relational Mapping — a layer that translates between in-memory objects and relational database rows, allowing persistent entities to be stored and retrieved without writing explicit SQL.

rejection manifest A data structure returned to the client after a replay pass, listing each command that could not be applied along with the reason for rejection and the current server-side state of the affected entity.

replay The process of re-executing a sequence of commands against an authoritative data store in order to synchronize state; in this thesis, the mechanism by which offline commands are submitted to the server on reconnection.

server-authoritative architecture A system design in which the server is the single source of truth for all persistent state; all state-modifying actions are validated and committed server-side, and the client cannot unilaterally alter authoritative data.

Service Locator A software design pattern that provides a centralized registry through which components can obtain references to services (such as a database interface) at runtime, without being coupled to specific implementations.

SQLite An embedded, serverless, ACID-compliant relational database engine implemented as a C library; it stores an entire database in a single file on disk and runs within the host process, requiring no separate server.

Steam Cloud Save Valve's file-level save synchronization service, which uploads and downloads save files when a player connects; conflicts are resolved by last-write-wins, discarding any changes made on the other device.

Telemetry Telemetry is the automated collection and transmission of data and measurements from distributed or remote sources to a central system for monitoring, analysis and resource optimization.

UUID Universally Unique Identifier — a 128-bit identifier standardized in RFC 4122, typically represented as a 36-character hyphen-separated hexadecimal string; used as idempotency keys to uniquely identify each command.

WAL Write-Ahead Logging — a SQLite journal mode in which changes are written to a separate log file before being applied to the main database, improving concurrent read performance and crash recovery.

References

Baker, N. (2025). *Online gaming statistics*. https://www.uswitch.com/broadband/studies/online-gaming-statistics/

Bernstein, P. A., & Goodman, N. (1981). Concurrency control in distributed database systems. *ACM Computing Surveys*, *13*(2), 185–221. https://doi.org/10.1145/356842.356846

Blizzard Entertainment. (2004). *World of warcraft*. https://worldofwarcraft.blizzard.com

Blizzard Entertainment. (2012). *Diablo III*. https://diablo3.blizzard.com

Céré, J., Montiglio, P.-O., & Kelly, C. D. (2021). Indirect effect of familiarity on survival: A path analysis on video game data. *Animal Behaviour*, *181*, 105–116. https://doi.org/https://doi.org/10.1016/j.anbehav.2021.06.010

Ellis, C. A., & Gibbs, S. J. (1989). Concurrency control in groupware systems. *ACM SIGMOD Record*, *18*(2), 399–407. https://doi.org/10.1145/66926.66963

Fowler, M. (2005). *Event sourcing*. https://martinfowler.com/eaaDev/EventSourcing.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, *33*(2), 51–59. https://doi.org/10.1145/564585.564601

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Hořeňovský, M., & catchorg contributors. (2023). *Catch2*. https://github.com/catchorg/Catch2

Iron Gate Studio. (2021). *Valheim*. https://www.valheimgame.com

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. https://doi.org/10.1145/3359591.3359737

Moll, P., Isak, S., Hellwagner, H., & Burke, J. (2021). A quadtree-based synchronization protocol for inter-server game state synchronization. *Computer Networks*, *185*, 107723. https://doi.org/https://doi.org/10.1016/j.comnet.2020.107723

Nystrom, R. (2014a). *Game programming patterns*. Genever — Benning. https://books.google.no/books?id=9fIwBQAAQBAJ

Nystrom, R. (2014b). *Service locator — game programming patterns*. https://gameprogrammingpatterns.com/service-locator.html

Ondruška, D., Scott, R., Katzner, M.-M., Cerezo, A. H., & Barczyk, M. (2026). *Stop killing games at the european parliament*. https://www.youtube.com/watch?v=QXdmoeaYZ9Y

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types, 386–400. https://doi.org/10.1007/978-3-642-24550-3_29

SQLite Consortium. (2024). *SQLite*. https://www.sqlite.org

Valve Corporation. (2023). *Steam cloud*. https://partner.steamgames.com/doc/features/cloud